



Rapport d'activité

Agence418, Le Soler
du 18 mai au 19 juin 2026

CodeGrimoire

Application web de gestion de snippets de code

Louka Kuhl

Réalisées lors du stage de 1ère année de BTS SIO1 spécialité SLAM

Tuteur de stage : Benoît Pascal — Agence418, Le Soler

Louka Kuhl — BTS SIO1 spécialité SLAM — Lycée Jean Lurçat — Promotion 2025/2026

Introduction

L'entreprise

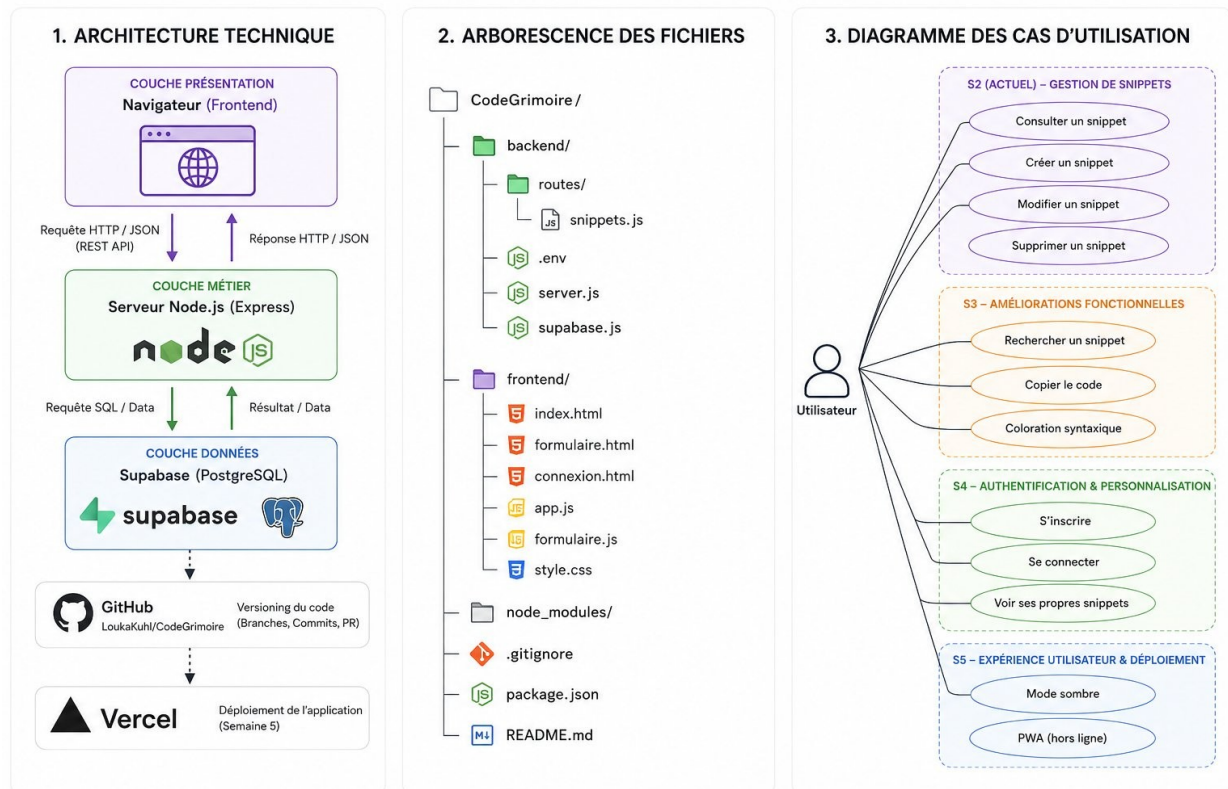
Agence418 est une agence web basée à Le Soler, spécialisée dans le développement d'applications web sur mesure et l'automatisation de processus métiers pour des PME et indépendants. L'entreprise est dirigée par Benoît Pascal, développeur fullstack indépendant et tuteur du stage.

Le projet

Mission : concevoir et développer CodeGrimoire, application web privée de stockage et recherche de snippets de code (filtrage par langage et mot-clé).

Stack technique

Composant	Technologie
Frontend	HTML5, CSS grimoire inline (thème écrit à la main), JavaScript ES6+ (vanilla)
Backend	Node.js v20.18.1 LTS, Express.js, TypeScript (migration semaine 3)
Base de données	Supabase (PostgreSQL)
Hébergement	Render (Docker Web Service backend + Static Site frontend)
Versionnement	Git + GitHub (repo public LoukaKuhl/CodeGrimoire)



Architecture technique, arborescence des fichiers et diagramme des cas d'utilisation

Réalisations

R1 - Mise en place de l'environnement de développement

Durée prévue : 1 jour | **Durée réelle :** 1 jour (lundi 18 mai 2026)

Technologies : Ubuntu 24.04, VS Code, Git, Node.js v20.18.1 LTS, npm 10.8.2

Mise à niveau du système d'exploitation Ubuntu 22.04 vers 24.04. Installation de l'ensemble de la chaîne d'outils : éditeur VS Code avec les extensions Prettier, ESLint, Tailwind CSS IntelliSense, Thunder Client et GitLens. Initialisation du dépôt Git local et création du repository privé CodeGrimoire sur GitHub. Mise en place de la structure de dossiers frontend/backend et envoi du premier commit.

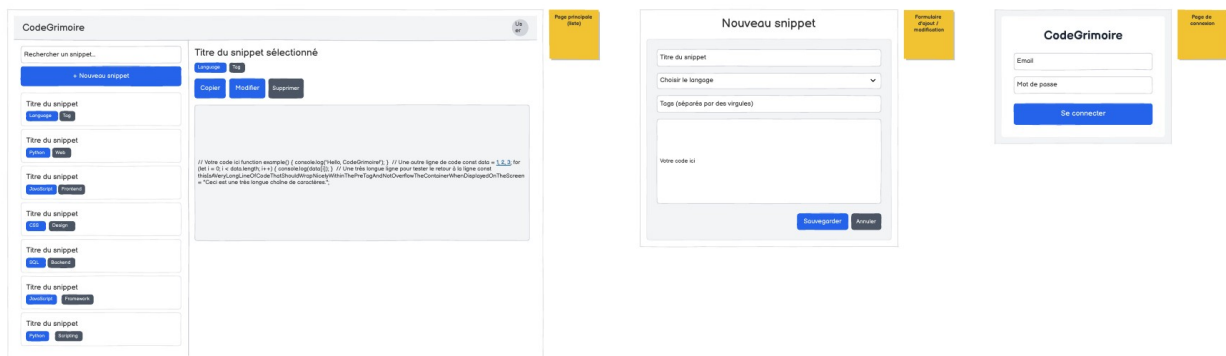
R2 - Conception

Durée prévue : 1 jour | **Durée réelle :** 1 jour (mardi 19 mai 2026)

Outils : Balsamiq Cloud (wireframes), Figma Make (maquette haute fidélité), DrawSQL (schéma BDD)

Maquettes fil de fer

Réalisation de trois écrans sur Balsamiq Cloud : liste des snippets, formulaire d'ajout/modification, page de connexion.



Maquettes Balsamiq Cloud - 3 écrans

Base de données

Création du projet Supabase. Conception de la table snippets avec les colonnes id (bigint, PK), title (text), code (text), language (text), tags (text?), created_at (timestamp?) et user_id (uuid?) pour la semaine 4. Configuration des clés API dans le fichier .env (exclu du versionnement via .gitignore).



Schéma de la base de données - tables snippets et users (DrawSQL)

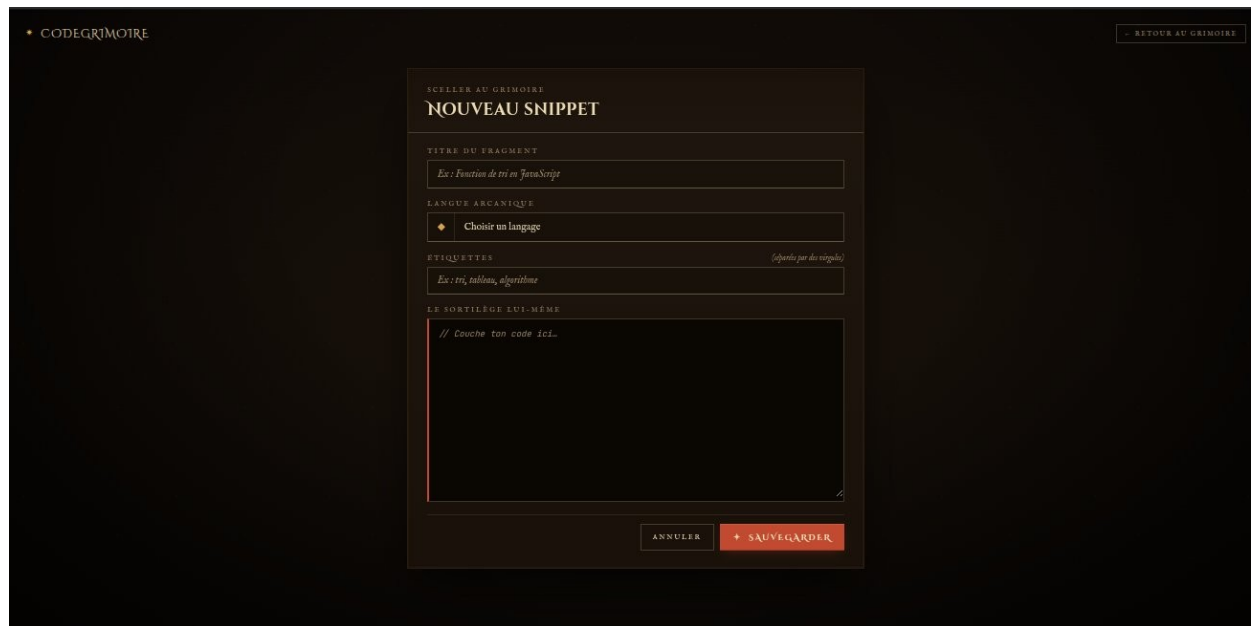
R3 - Développement frontend - Structure HTML

Durée prévue : 2 jours | **Durée réelle :** 3 jours (mercredi 20 au vendredi 22 mai 2026)

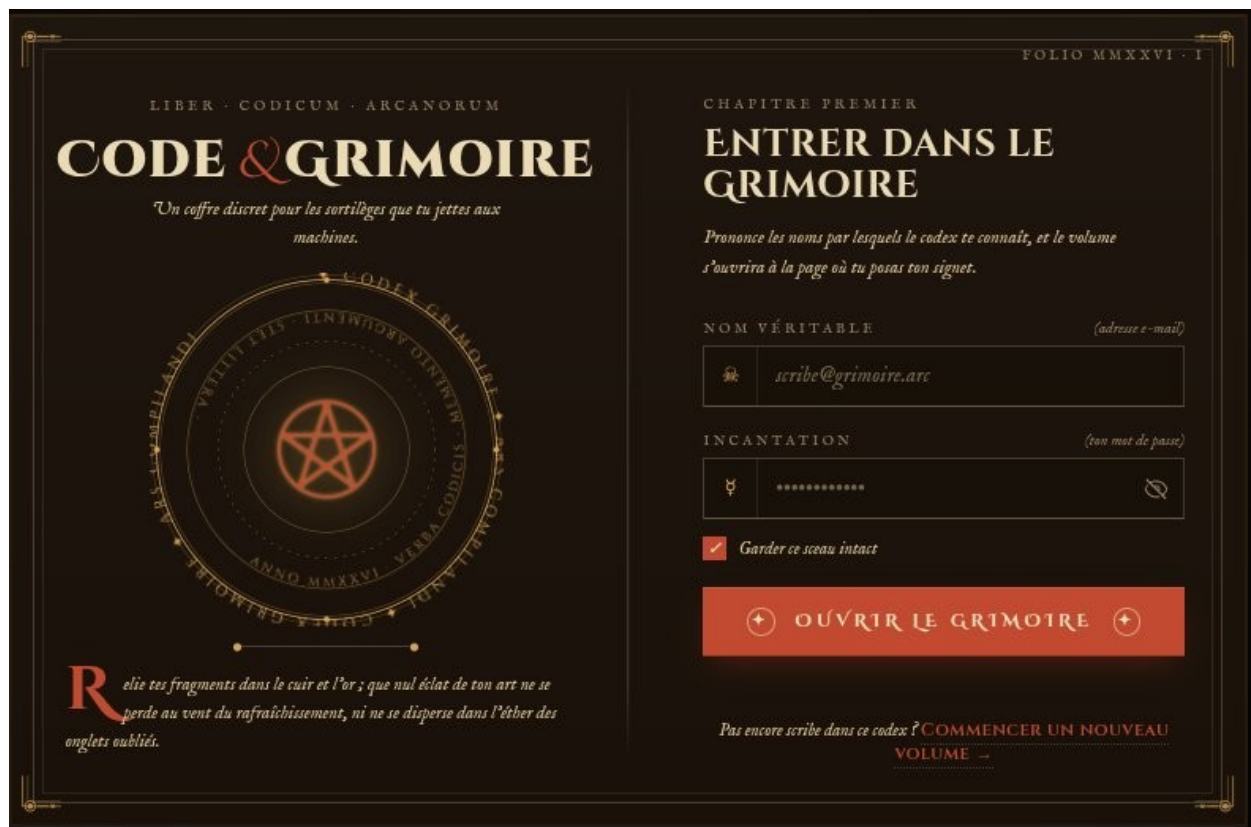
Technologies : HTML5, CSS inline (base Tailwind CDN intégrée en semaine 1, remplacée par CSS grimoire écrit à la main dès la semaine 2)

Construction des trois pages de l'application :

- **index.html** : page principale avec navbar, sidebar (barre de recherche + liste des snippets avec badges de langage) et zone de détail (titre, code, boutons Copier / Modifier / Supprimer)
- **formulaire.html** : formulaire de création et modification (champs titre, langage, tags, zone de code, boutons Sauvegarder et Annuler)
- **connexion.html** : formulaire de connexion (email, mot de passe, liens Mot de passe oublié et Créer un compte)



Formulaire de création d'un snippet après reskin - thème grimoire



Page de connexion après reskin - thème grimoire

R4 - Développement backend - API REST

Durée prévue : 2 jours | **Durée réelle :** 3 jours (vendredi 22 - mardi 26 mai 2026)

Technologies : Node.js, Express.js, Supabase JS, dotenv, cors, ws

Serveur Express dans server.js. Connexion Supabase dans supabase.js dédié (évite les imports circulaires).

Cinq routes CRUD dans routes/snippets.js :

Méthode	Route	Description
GET	/snippets	Récupération de tous les snippets
GET	/snippets/:id	Récupération d'un snippet par identifiant
POST	/snippets	Création d'un nouveau snippet (201 Created)
PUT	/snippets/:id	Modification d'un snippet existant
DELETE	/snippets/:id	Suppression d'un snippet

Tests API : Validation de chaque route via Thunder Client (GET : 200 OK, POST : 201 Created en 108 ms sur localhost:3000).

Problèmes rencontrés : Trois bugs consécutifs résolus : variable .env mal nommée (SUPABASE_SECRET_KEY), incompatibilité WebSocket Node.js 20 (installation du package ws), import circulaire entre server.js et snippets.js (résolu par la création de supabase.js).

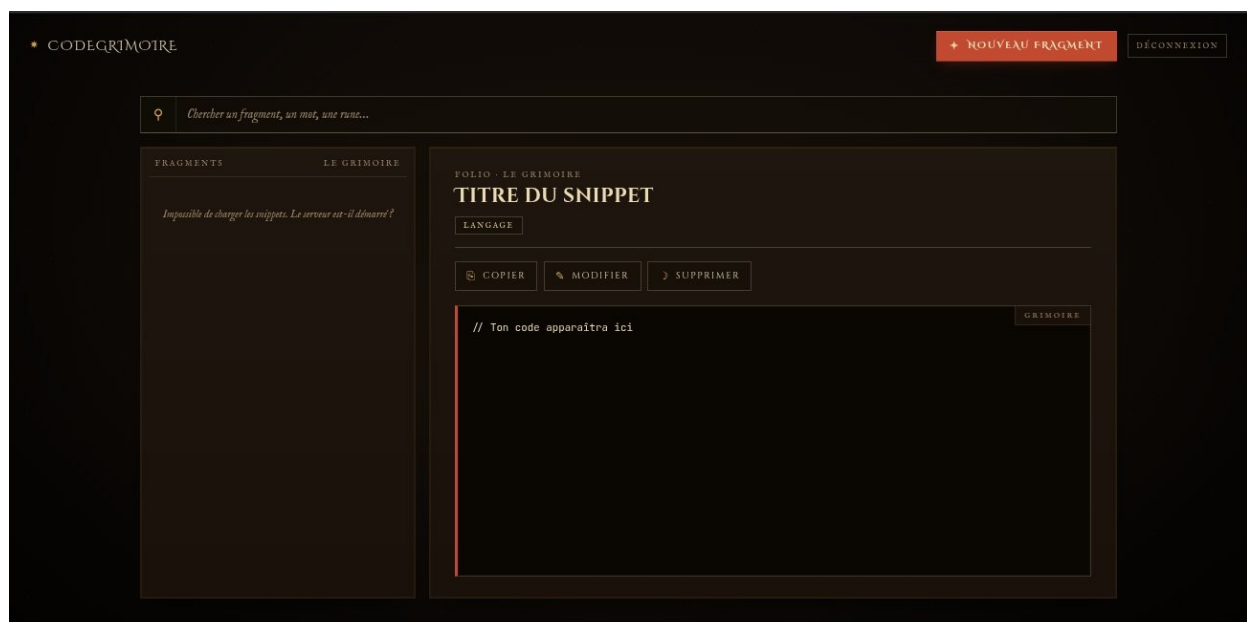
R5 - Développement frontend - JavaScript

Durée prévue : 2 jours | **Durée réelle :** 3 jours (mardi 26 au mercredi 27 mai 2026)

Technologies : JavaScript ES6+, API Fetch, API Clipboard

Fichiers app.js et formulaire.js assurant la liaison HTML / API REST :

- chargerSnippets() : appel GET à l'API au chargement de la page, stockage en mémoire dans tousLesSnippets
- afficherSnippets() : génération dynamique du HTML de la sidebar avec badges de langage colorés
- afficherDetail() : affichage du snippet sélectionné dans la zone de détail (mise à jour de #detail-titre, #detail-code et #badge-langage)
- supprimerSnippet() : appel DELETE + rechargement de la liste
- formulaire.js : détection de l'id en URL pour distinguer création (POST) et modification (PUT)



Problème résolu : Après suppression, dataset.id persistait dans le DOM, provoquant une tentative de PUT sur un id inexistant. Correction : ajout de `delete document.getElementById('detail-snippet').dataset.id` dans `supprimerSnippet()`.

Rédaction CONVENTIONS.md v2.0 (jeudi 28 mai)

Rédaction itérative du fichier docs/CONVENTIONS.md en 10 versions (v1.0 à v2.0), 18 sections. Contenu : paradigme procédural obligatoire, CommonJS backend, scripts frontend sans framework, async/await obligatoire (`.then()` interdit), `===` obligatoire, pas de `var`, opérateur `?.` et `??` préférés, ESLint sans Prettier. Document révisé avec Benoît Pascal et commité sur GitHub.

Nettoyage et documentation du code (vendredi 29 mai)

- Ajout de commentaires JSDoc sur toutes les fonctions de `app.js` et `formulaire.js` : paramètres, valeur de retour, comportement attendu

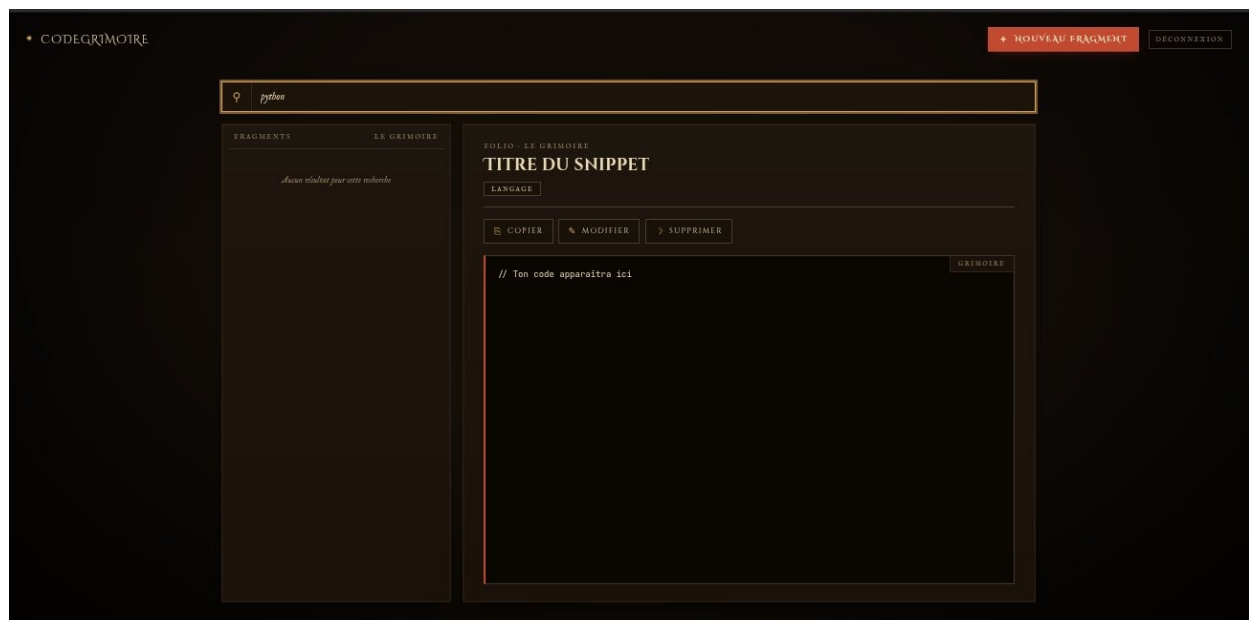
R6 - Fonctionnalité de recherche

Durée prévue : 1 jour | **Durée réelle :** 1 jour (lundi 1er juin 2026)

Technologies : JavaScript ES6+, filtrage en mémoire

Implémentation d'un filtre en temps réel sur la barre de recherche. Écoute sur l'événement `input`, comparaison du texte saisi avec les champs `title` et `language` de chaque snippet en mémoire. Mise à jour dynamique de l'affichage sans appel réseau supplémentaire.

Tests réalisés : 3 scénarios validés : filtre par langage ('python', 'css') et filtre par mot-clé dans le titre ('boucle').



Recherche en temps réel après reskin - filtre 'python' et message 'Aucun résultat'

Corrections CONVENTIONS.md (lundi 1er juin)

Revue du CONVENTIONS.md v2.0 avec le tuteur. Cinq corrections appliquées :

- Conflit onclick vs addEventListener tranché : delegation via addEventListener privilégiée, onclick toléré uniquement pour les cas simples documentés
- Exception classe d'erreur explicitée : ValidationError et NotFoundError autorisées malgré le paradigme procédural
- IDs HTML manquants documentés : ajout de #detail-titre, #detail-code dans la section éléments DOM
- Référence à getBadge() supprimée (fonction retirée du code mais restée dans la doc)
- Table des matières complétée : section 6 ajoutée

R7 - Bouton Copier

Durée prévue : 0,5 jour | **Durée réelle :** 0,5 jour (mardi 2 juin 2026)

Technologies : API Clipboard (navigator.clipboard.writeText)

Au clic sur le bouton Copier, le contenu du snippet affiché est copié dans le presse-papier via l'API Clipboard. Retour visuel temporaire sur le bouton (changement de texte) pour confirmer l'action.

R8 - Correction et qualité du code

Durée prévue : 1 jour | **Durée réelle :** 1,5 jour (mardi 2 et mercredi 3 juin 2026)

Deux incohérences corrigées dans formulaire.js lors de la revue de code :

- API_URL : alignement sur la détection dynamique (localhost/production) déjà présente dans app.js
- Ligne 29 : remplacement de == par === (opérateur d'égalité stricte) conformément aux conventions

Alerte sécurité Supabase : Réception d'une alerte rls_disabled_in_public signalant que la table snippets est accessible sans authentification. Risque limité à ce stade (backend local, clé non publiée). Correction apportée en semaine 4 : activation du RLS, 4 policies et colonne user_id (voir R14).

UX - Message 'Aucun résultat' : Ajout d'un message dédié dans la sidebar quand aucun snippet ne correspond à la recherche saisie.

JavaScript - Paramètres par défaut : Application des paramètres par défaut ES6+ (function maFonction(param = valeur)) dans le code de CodeGrimoire pour renforcer la robustesse des fonctions frontend.

Frontend - Affichage des tags : Les tags étaient présents en base de données mais non rendus dans l'interface. Ajout de leur affichage dans la zone de détail via getElementById et textContent, en dessous du titre du snippet.

R9 - Journée qualité et outillage

Durée prévue : 1 jour | **Durée réelle :** 1 jour (jeudi 4 juin 2026)

Technologies : ESLint, Node.js, Git

Mise en place complète de la chaîne qualité sur le projet :

- ESLint : création de eslint.config.js, ajout du script lint dans package.json, correction du champ main pointant vers un fichier index.js inexistant

- CLAUDE.md : création du fichier de référence pour Claude Code (stack, conventions, architecture attendue, commandes)
- Refactoring backend : extraction de la logique Supabase de routes/snippets.js vers backend/services/snippets.service.js. Les routes restent fines (HTTP uniquement). Suite de vérification complète : syntaxe, chargement des modules, ESLint, tests CRUD manuels dans le navigateur — tous passés.
- Suppression des commentaires ligne par ligne dans les fichiers JS (réservés à la documentation, pas au code de production)

Audit de conformité : Analyse du code réel via fetch des fichiers bruts GitHub. Problèmes identifiés et priorisés : vulnérabilité XSS dans afficherSnippets() (innerHTML sans echapperHtml()), commentaires ligne par ligne encore présents dans connexion.html et formulaire.html, incohérences onclick vs addEventListener. Ces points ont constitué la feuille de route du vendredi 5 juin.

R10 - Sécurité, nettoyage et migration TypeScript

Durée prévue : 1 jour | **Durée réelle :** 1 jour (vendredi 5 juin 2026)

Technologies : HTML, JavaScript, TypeScript, Node.js

Trois axes de travail distincts réalisés en une journée :

Nettoyage des commentaires HTML

- Suppression des commentaires ligne par ligne dans connexion.html et formulaire.html (réservés à la documentation, pas au code de production, conformément à CONVENTIONS.md)

Protection XSS

- Ajout de la fonction echapperHtml() dans app.js : protection XSS par echappement des caracteres speciaux HTML avant injection dans le DOM.
- Application de echapperHtml() sur les champs title et code dans afficherSnippets() et afficherDetail()

Objectif : Prévenir les attaques Cross-Site Scripting (XSS) en s'assurant qu'aucune donnée issue de la base ne peut être interprétée comme du HTML par le navigateur.

Migration TypeScript du backend

À la demande du tuteur Benoît Pascal, migration du backend Node.js de JavaScript vers TypeScript :

- Installation des packages TypeScript : typescript, @types/node, @types/express, @types/cors, @types/ws, tsx
- Création de tsconfig.json (cible uniquement le backend)
- Renommage des fichiers backend .js en .ts
- Audit de cohérence de la migration

Impact planning : Migration non prévue au planning initial mais validée par le tuteur. Le comportement de l'API reste identique.

R11 - Sécurisation de l'API

Durée prévue : 1 jour | **Durée réelle :** 1 jour (lundi 8 juin 2026)

Technologies : TypeScript, helmet, cors, express-rate-limit

Mise en place des middlewares de protection sur la branche feature/securite-api :

- helmet : ajout automatique des en-têtes HTTP de sécurité (X-Content-Type-Options, X-Frame-Options, etc.)
- CORS configurable via variable d'environnement CORS_ORIGIN : permissif en développement local, verrouillable pour la production
- express-rate-limit : limitation à 100 requêtes par fenêtre de 15 minutes par IP
- Gestionnaire JSON 404 : toute route non définie renvoie {erreur: 'Route introuvable'} au lieu d'une page HTML
- Helper messageErreur() extrait dans backend/utlis/messageErreur.ts pour réutilisation dans le middleware central
- ESLint mis à jour avec argsIgnorePattern: '^_' pour le paramètre _next intentionnellement non utilisé

Vérification : 7 points de contrôle passés : build TypeScript, lint, GET /snippets, route 404, en-têtes helmet, en-têtes rate-limit, round-trip POST/DELETE.

Corrections d'audit finalisées : README.md mis à jour, .editorconfig créé (indentation 4 espaces, 2 pour JSON, LF, UTF-8, limite 100 caractères), CONVENTIONS.md porté à v5.1 (section qualité nettoyée, codes HTTP corrigés, erreurs typées différées à S4).

R12 - Validation des données et erreurs typées

Durée prévue : 1 jour | **Durée réelle** : 1 jour (lundi 8 juin 2026)

Technologies : TypeScript, Node.js, Express

Mise en place d'une couche de validation et d'une gestion d'erreurs structurée :

- Création de backend/erreurs.ts : classes ValidationError (400) et NotFoundError (404) étendant Error, avec message et statusCode
- Fonction validerDonneesSnippet() : valide la présence et le type des champs title, language, code avant toute insertion ou modification en base
- Détection des ressources inexistantes via .select() après les opérations Supabase : si data.length === 0 → lève NotFoundError
- Middleware central d'erreur Express (4 paramètres : erreur, req, res, _next) : intercepte toutes les erreurs, renvoie le bon code HTTP et un message JSON structuré (400/404/500)

Tests : 10 scénarios validés (création valide, champs manquants → 400, id inexistant → 404, modification → 200, suppression → 200, round-trip complet).

Documentation : CONVENTIONS.md mis à jour en v5.2 : ajout de la section gestion des erreurs typées (ValidationError, NotFoundError, middleware central).

R13 - Authentification des utilisateurs

Durée prévue : 1 jour | **Durée réelle** : 1 jour (mercredi 10 juin 2026)

Technologies : Supabase Auth, JavaScript, HTML, ESLint

Mise en place de l'authentification

- Développement des pages connexion.html et inscription.html avec intégration de Supabase Auth : connexion via signInWithPassword() et inscription via signUp() par email et mot de passe
- Gestion de session : vérification de la session au chargement de chaque page protégée, redirection automatique des visiteurs non connectés vers connexion.html
- Ajout de la déconnexion (signOut()) et de l'option 'se souvenir de moi' (persistance de session configurable)

Améliorations issues de l'audit de pré-production

- Affichage/masquage du mot de passe (toggle input type password/text)
- Redirection automatique vers index.html si l'utilisateur est déjà connecté
- Suppression d'un lien inactif dans la navigation
- Messages d'erreur explicites en cas d'échec d'authentification (email inconnu, mot de passe incorrect, compte déjà existant)
- Protection contre la double soumission : désactivation du bouton pendant le traitement de la requête
- Association des libellés HTML aux champs de formulaire (attributs for/id) pour l'accessibilité
- Message de confirmation affiché après une inscription réussie
- Suppression de l'affichage transitoire du contenu avant vérification de session (FOUC) sur les pages protégées

Conformité : Mise en conformité du code frontend avec les règles ESLint du projet.

R14 - Sécurité base de données : RLS, user_id et erreur 401

Durée prévue : 1 jour | **Durée réelle :** 1 jour (jeudi 11 juin 2026)

Technologies : Supabase PostgreSQL, SQL, TypeScript, Express

Row Level Security et colonne user_id

Exécution des commandes SQL dans le SQL Editor de Supabase pour sécuriser l'accès multi-utilisateur à la table snippets :

- Ajout de la colonne user_id (uuid, référence auth.users(id), valeur par défaut auth.uid()) : chaque snippet est désormais automatiquement associé à l'utilisateur connecté lors de la création
- Activation du Row Level Security sur la table snippets
- Création de 4 politiques garantissant que chaque utilisateur n'accède qu'à ses propres snippets : SELECT (using auth.uid() = user_id), INSERT (with check auth.uid() = user_id), UPDATE (using auth.uid() = user_id), DELETE (using auth.uid() = user_id)
- Sauvegarde du SQL dans backend/db/rls-snippets.sql et commit dans le dépôt pour traçabilité

Gestion de l'erreur 401

- Mise à jour du middleware central d'erreur dans backend/server.ts : les erreurs d'authentification Supabase (code PGRST301, token invalide ou expiré) renvoient désormais un HTTP 401 au lieu d'un 500 générique

R15 - Migration onclick/addEventListener et début du reskin

Durée prévue : 1 jour | **Durée réelle :** 1 jour (vendredi 12 juin 2026)

Technologies : JavaScript, HTML, CSS grimoire inline, Google Fonts

Migration onclick vers addEventListener

Remplacement de tous les gestionnaires d'événements inline (attributs onclick="") par des écouteurs addEventListener() dans app.js et formulaire.js, conformément aux conventions CONVENTIONS.md. La délégation d'événements est désormais appliquée sur l'ensemble du frontend.

Résultat lint : 0 erreur ESLint après migration (seul avertissement résiduel attendu : enTetesAuth). Commit 3198305 sur main.

Reskin grimoire complet

- Chargement de la police Cinzel (Google Fonts, poids 400 et 600) dans l'en-tête de index.html
- Application de Cinzel au logo CodeGrimoire de la navbar et au titre du snippet dans la zone de détail
- Passage des blancs purs vers un blanc parchemin chaud (#ECE6D8) sur les titres

Reskin complet (vendredi après-midi) : Extension du thème grimoire aux 4 pages (index.html, connexion.html, inscription.html, formulaire.html). Corrections d'audit accessibilité (attributs aria, garde de session renforcée, documentation du token). CONVENTIONS.md porté à v5.3 (sections auth et RLS actées, lien README corrigé). Détection dynamique de l'environnement pour API_URL dans app.js et formulaire.js (ligne de production = placeholder à remplacer au déploiement).

R16 - Dockerisation du backend

Durée prévue : 0,5 jour | **Durée réelle** : 0,5 jour (vendredi 12 juin 2026)

Technologies : Docker, Node.js, TypeScript

- Création d'un Dockerfile multi-stage : étape build (tsc compile le TypeScript vers dist/) et étape run (image Node alpine, copie uniquement dist/ et node_modules de production)
- Création du .dockerignore excluant node_modules, src TypeScript, fichiers de développement
- Build et test en local : conteneur démarré, connexion Supabase et routes CRUD vérifiées via le conteneur

Objectif : Produire une image portable et reproductible pour le déploiement sur un hébergeur container (Render).

R17 - Déploiement sur Render

Durée prévue : 1 jour | **Durée réelle** : 1 jour (lundi 15 juin 2026)

Technologies : Render, Docker, GitHub

URLs de production : Backend : codegrimoire.onrender.com | Frontend : codegrimoire-frontend.onrender.com

Déploiement

- Backend déployé sur Render en tant que Web Service Docker : connexion au dépôt GitHub, détection automatique du Dockerfile, configuration des variables d'environnement (SUPABASE_URL, SUPABASE_PUBLISHABLE_KEY, CORS_ORIGIN)

- Frontend déployé sur Render en tant que Static Site : répertoire de publication frontend/, URL du backend renseignée dans API_URL
- CORS_ORIGIN affiné avec l'URL exacte du frontend Render

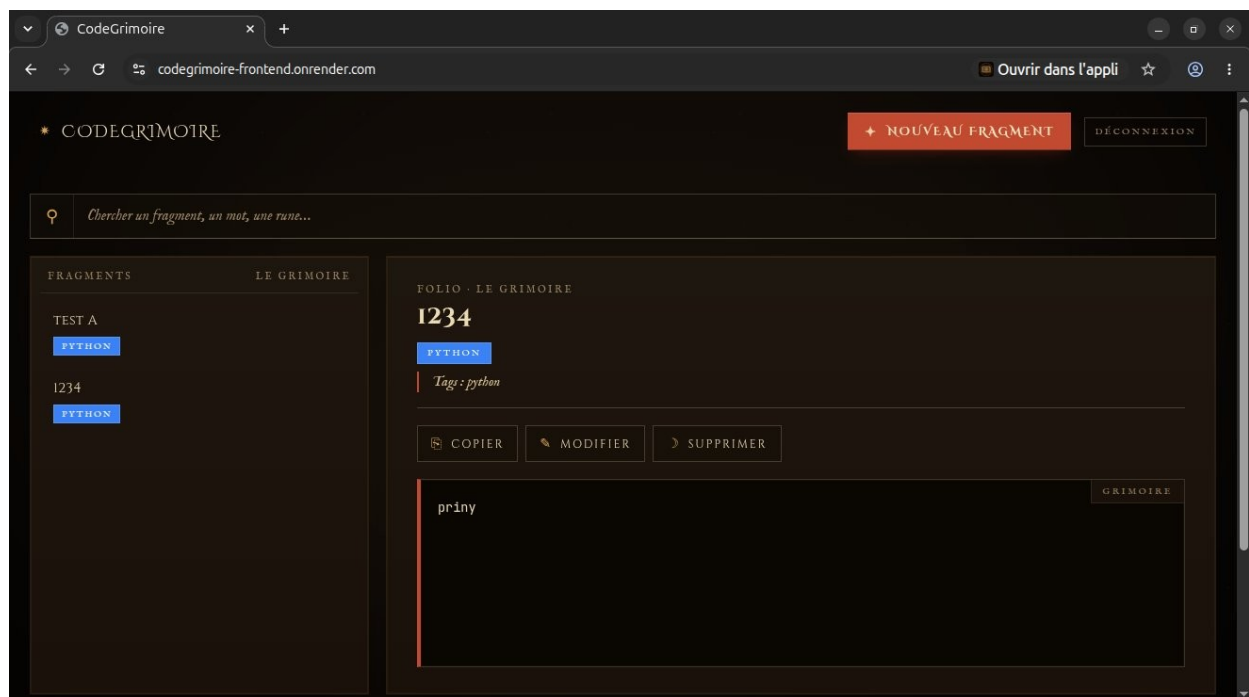
Corrections post-déploiement

- Configuration du trust proxy dans server.ts pour le rate-limiting derrière le reverse proxy Render
- Fermeture du CORS par défaut (origin: false) quand CORS_ORIGIN est absente
- Ajout des retours d'erreur utilisateur sur la suppression de snippet et le chargement du formulaire de modification
- Suppression de style.css vide et non référencé
- Mise à jour de CONVENTIONS.md section déploiement et du GUIDE-CodeGrimoire.md (sections Render et comportements API)

Tests et validation

Tests automatisés : Campagne backend 12/12 PASS, 0 FAIL : sécurité (401 sans token), validation (400 sur champs manquants), routage (404 sur route inexistante), CRUD complet.

Audit de conformité : 17/17 points validés : build TypeScript, lint ESLint, git, code, documentation et déploiement tous conformes.



Application CodeGrimoire déployée sur Render (codegrimoire-frontend.onrender.com) — PWA installable visible dans la barre Chrome

R18 - Progressive Web App

Durée prévue : 0,5 jour | **Durée réelle** : 0,5 jour (mardi 16 juin 2026)

Technologies : Service Worker API, Cache API, JavaScript

Transformation de CodeGrimoire en Progressive Web App permettant l'installation sur mobile et l'accès hors ligne :

- Création de frontend/sw.js : gestionnaire install (mise en cache initiale de 14 fichiers : HTML, JS, CSS, icônes), gestionnaire activate (purge des anciens caches via `cache.keys()`), gestionnaire fetch (stratégie cache-first : réponse depuis le cache si disponible, sinon requête réseau)
- Nom du cache : `codegrimoire-v2` (incrémenté pour forcer le remplacement du cache précédent)
- Enregistrement du Service Worker dans `index.html` via `navigator.serviceWorker.register()`

Validation : ESLint 0 erreur après intégration. Vérification du cycle install/activate dans les DevTools. Projet déclaré fonctionnellement complet à l'issue de cette réalisation.

R19 - Revue finale et alignement de la documentation

Durée prévue : 1 jour | **Durée réelle :** 1 jour (vendredi 19 juin 2026)

Technologies : JavaScript, HTML, ESLint

Audit de conformité final du projet avant clôture du stage :

- Identification que Tailwind CSS (intégré via CDN en semaine 1) avait été remplacé par du CSS grimoire écrit à la main dès la semaine 2, sans jamais être explicitement supprimé
- Suppression des classes Tailwind mortes dans `app.js` (classes référencées dans le code JS mais sans effet faute de CDN)
- Alignement de la documentation sur le code réel : retrait des mentions Tailwind dans `README.md`, `CONVENTIONS.md` et `GUIDE-CodeGrimoire.md`

Résultat : Documentation conforme au code source. Aucune mention technique inexacte dans le projet livré.

Organisation

Méthode de travail

Planning : Découpage en 5 semaines thématiques : S1 Conception et environnement, S2 CRUD complet, S3 Recherche et copie, S4 Authentification, S5 Mode sombre, PWA et déploiement.

Suivi quotidien : Point en fin de journée avec le tuteur, validation des objectifs de la journée avant passage à la suivante.

Versionnement : Commits quotidiens avec messages explicites. Branches dédiées pour les correctifs (ex : `fix/formulaire-coherence`). Dépôt privé sur GitHub : [LoukaKuhl/CodeGrimoire](#).

Documentation

CONVENTIONS.md : Document de conventions de développement élaboré de façon itérative (10 versions, v1.0 à v2.0, 18 sections). Couvre : paradigme procédural obligatoire, CommonJS côté backend, `async/await` obligatoire (`.then()` interdit), ESLint sans Prettier, `===` obligatoire, interdiction de `var`.

README.md : Description du projet, stack technique, procédure d'installation locale, structure des fichiers.

Outils et contraintes

Environnement de développement

Outil	Usage
VS Code	Éditeur principal avec extensions ESLint, Tailwind CSS IntelliSense, GitLens, Thunder Client
Git / GitHub	Versionnement, branches de correctifs, dépôt public
Thunder Client	Tests des routes API REST (GET, POST, PUT, DELETE)
Balsamiq Cloud	Wireframes fil de fer
Figma Make	Maquette haute fidélité (palette, typographie, espacements)
Supabase	Base de données PostgreSQL + Auth + tableau de bord d'administration
Vercel	Déploiement continu depuis GitHub (semaine 5)

Qualité du code

- ESLint configuré sur le projet avec script lint dans package.json
- CONVENTIONS.md v5.3 définissant les règles de codage appliquées sur l'ensemble du projet
- 12 cas de test automatisés (12/12 PASS) : sécurité, validation, routage et CRUD complet (création, lecture, modification, suppression, cas limites)

Sécurité et cadre juridique

- Clés API Supabase stockées dans le fichier .env, non versionné (.gitignore)
- Row Level Security activée sur la table snippets avec 4 politiques (R14) : Row Level Security désactivée sur la table snippets. Correction planifiée en semaine 4 (activation RLS + politiques + colonne user_id)
- Authentification JWT via Supabase Auth implémentée en semaine 4 (R13, R14)
- RGPD : l'application est privée et à usage personnel. Aucune donnée à caractère personnel de tiers n'est collectée. Les clés d'accès Supabase ne sont pas versionnées ni exposées publiquement.
- Confidentialité : les captures d'écran et codes sources partagés dans ce rapport ne contiennent aucune information sensible (clés API masquées, données fictives utilisées pour les tests).

Conclusion

CodeGrimoire est une application web fullstack complète, déployée en production sur Render depuis le 15 juin 2026. Elle couvre l'intégralité du cycle de développement : conception (maquettes, schéma BDD), développement frontend et backend, sécurisation (helmet, CORS, rate-limit, RLS, authentification JWT), qualité (ESLint, TypeScript, tests automatisés 12/12), et déploiement continu (Docker + Render). Le projet est accessible à l'adresse codegrimoire-frontend.onrender.com.

Points positifs

- Projet fullstack complet conçu, développé et déployé en production en 5 semaines

- Démarche qualité intégrée dès le départ (conventions, commits, branches)
- Résolution autonome de problèmes techniques réels (bugs Supabase, import circulaire, casse de table)

Points à améliorer

- Mise en place des tests automatisés plus tôt dans le projet (ils sont arrivés en semaine 5)
- Prévoir la colonne user_id dans le schéma initial pour éviter la migration en semaine 4